Assignment 3 – CO3

Constructors, Destructors, Copying and Operator Overloading

Field	Details
Name / Register No.	Kaviya V/ 192421022
GitHub repository	https://github.com/kaviyavellaisamy/DSA0107---C-/tree/main

Every question follows Design → Implementation → Analysis → Evaluation. Programs were compiled with g++ -std=c++17 -Wall -Wextra; sanitizer runs add -fsanitize=address (and undefined). Timings are from one machine with -O2 unless stated.

Q1 – Constructors and Constructor Overloading (6 marks)

Design

Situation	Constructor	State after construction
Empty record, filled later	Book() – delegates to the full constructor	title "Untitled", author "Unknown", copies 0
Donated book, only a title	explicit Book(const string& t) – delegates	author "Unknown", copies 1
New book, full details	Book(t, a, copies) – the only one that checks	copies must be ≥ 0, else std::invalid_argument

```
class Book {
    std::string title_, author_;
    int copies_;
    static int checked(int c) { if (c < 0) throw std::invalid_argument("copies must be >= 0");
return c; }
public:
    Book() : Book("Untitled", "Unknown", 0) {}                      // delegation
    explicit Book(const std::string& t) : Book(t, "Unknown", 1) {}  // delegation
    Book(const std::string& t, const std::string& a, int c)
        : title_(t), author_(a), copies_(checked(c)) {}             // initializer list +
single check
};
```

Because the two short constructors delegate to the full one, the number of copies is validated in **one** place, and no Book object can exist with a negative count: the exception leaves the constructor before the object is formed.

Analysis

- **Compiler-supplied default constructor:** the compiler writes Book() only if the class declares no constructor at all. As soon as any constructor exists (even a parameterised one) the implicit default is not generated; Book b; or Book arr[3]; then fail unless one is written (or = default is used).

- **Initializer list versus assignment in the body:** a member is always initialised before the body runs. With assignment in the body, std::string members are first default-constructed and then assigned (two steps); const members and reference members cannot be assigned at all, so the list is mandatory for them; base classes and member objects without a default constructor must be initialised in the list.
- **Order of initialisation:** members are initialised in the order they are declared in the class, not the order they appear in the list. The -Wreorder warning exists for this reason (experiment below).

State of the three objects after construction

```
empty      : [title="Untitled"      | author="Unknown"       | copies=0 ]
donated    : [title="Wings of Fire" | author="Unknown"       | copies=1 ]
full       : [title="Malgudi Days"  | author="R. K. Narayan" | copies=12]
sizeof(Book) = 72 bytes (two std::string of 32 bytes each + int + padding)
```

Experimental investigation (actual results)

```
Untitled | Unknown | copies=0
Wings of Fire | Unknown | copies=1
Malgudi Days | R. K. Narayan | copies=12
rejected: copies must be >= 0                    <- Book("Bad","Nobody",-3)
array of 3 Books created with the default constructor
```

Experiment	Observed
Remove the default constructor, create Book shelf[3];	error: no matching function for call to 'Book::Book()'
Wrong order: struct S { int a; int b; S(int x) : b(x), a(b + 1) {} };	warning: 'S::b' will be initialized after 'int S::a' [-Wreorder]. a is initialised first, from an indeterminate b (printed a=1, b=10) – the result depends on garbage.
Negative copies: Book("Bad","Nobody",-3)	std::invalid_argument caught: "copies must be >= 0"; no object created

Design question – when should a constructor be explicit?

Mark every single-argument constructor explicit unless an implicit conversion is really intended: otherwise the compiler silently converts a string into a Book wherever a Book is expected. Test program: void shelve(Book); shelve(std::string("Dune")); Book b = std::string("Emma");

Version	Result
Book(const std::string&) without explicit	Compiles: a temporary Book is made from the string, probably by mistake.
explicit Book(const std::string&)	error: could not convert … from 'std::string' to 'Book'; error: conversion from 'std::string' to non-scalar type 'Book' requested. Explicit calls such as Book("Dune") still work.

Alternative concept – one constructor with default arguments

```
Book(const std::string& t = "", const std::string& a = "Unknown", int c = 0);   // one
function
```

Output: Book a → [] Unknown 0; Book b("Dune") → [Dune] Unknown 0; Book c("Dune","Herbert",5) → [Dune] Herbert 5. Time and space of creation are O(1) in both designs (sizeof 72 bytes either way). **Where the copies are checked:** both designs check in one place (the full/defaulted constructor) – the overloaded version only because of delegation; overloads without delegation would repeat the check three times. **Difference in behaviour:** with default arguments Book("Dune") gets 0 copies (the default for "empty"), so the rule "a donated book has 1 copy" cannot be expressed without a sentinel value; overloads express the three situations as three named intentions.

Q2 – Copy Constructors: Shallow and Deep Copies (7 marks)

Design – memory diagrams

```
SHALLOW (compiler-generated copy: copies the pointer only)

  p: [ n=2 | medicines ]---+
                           +--->[ "Paracetamol" | "ORS" ]   one array, two owners
  q: [ n=2 | medicines ]---+

  q.set(1,...) changes p too.  At scope end both destructors run delete[] on the same array:
double delete.

DEEP (copy constructor allocates and copies)

  p: [ n=2 | medicines ]---->[ "Paracetamol" | "ORS" ]
  q: [ n=2 | medicines ]---->[ "Paracetamol" | "ORS" ]   separate arrays, independent
```

Implementation – Complete the Code (TODO 1 to 3)

```
Prescription(const Prescription& o)                        // TODO 1: deep copy constructor
    : n(o.n), medicines(new std::string[o.n]) {
    for (int i = 0; i < n; ++i) medicines[i] = o.medicines[i];
}
Prescription& operator=(const Prescription& o)             // TODO 2: copy assignment
{
    if (this != &o) {                                      // guard against p = p
        std::string* t = new std::string[o.n];             // build the new array first
        for (int i = 0; i < o.n; ++i) t[i] = o.medicines[i];
        delete[] medicines;                                // then release the old one
        medicines = t; n = o.n;
    }
    return *this;
}
~Prescription() { delete[] medicines; }                    // TODO 3
```

Actual output under -fsanitize=address (identical to expected output, sanitizer reports nothing, exit code 0)

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

Analysis

- **Why the shallow version crashes:** with no copy constructor the compiler copies member by member, so q.medicines holds the same address as p.medicines. At the end of main, q's destructor deletes the array, and then p's destructor deletes it again (double delete, undefined behaviour). Until then the program also behaves wrongly: q.set(1, "Cetirizine") changes p.
- **Rule of Three:** if a class needs a user-written destructor, copy constructor or copy assignment operator, it almost certainly needs all three, because all three deal with the same owned resource. The destructor releases it, the copy constructor and the assignment must each create their own copy so that every owner has exactly one array to release.
- **The three situations in which the copy constructor is called:** (1) initialisation from another object: Prescription q = p; (2) passing an object by value to a function; (3) returning an object by value (unless the copy is elided). Note that r = p is NOT a copy construction – it calls the copy assignment operator on an existing object.

Experimental investigation (actual results)

Experiment	Observed
Copies made when a Prescription is passed to f(Prescription) vs f(const Prescription&)	by value: 1 copy; by const reference: 0 copies
Independence: q = p then q.set(0,"Aspirin"); r = p	p[0]=Paracetamol q[0]=Aspirin r[0]=Paracetamol – independent
Sanitizer, version WITHOUT the copy constructor	ERROR: AddressSanitizer: heap-use-after-free … in Prescription::~Prescription() – the second destructor touches strings inside the array already freed by the first (a double free would follow)
Sanitizer, deep-copy version	no error, exit code 0
Self-assignment s = s WITH the guard	after s = s: Paracetamol ORS (unchanged)
Self-assignment s = s WITHOUT the guard (delete[] first, then copy)	after s = s: (blank) – the medicines are silently erased: no crash and no sanitizer error, so this bug is easy to miss

Time to copy a prescription (average per copy, µs, -O2)

n medicines	Deep copy (array)	vector<string> (compiler copy)	Shallow copy (pointer only)
10	0.104	0.074	0.0007
1,000	9.02	5.69	0.0007
100,000	1,134	599	0.0007

Deep copy grows linearly, 10 → 1,000 → 100,000 gives ≈ ×87 and ×126 (O(n)); the shallow copy stays constant (O(1)) – which is exactly why it is fast and why it is wrong.

Conclusion – when can you rely on the compiler-generated copy constructor?

When every member is itself safe to copy member-wise: built-in values, std::string, std::vector, other classes that copy deeply (Rule of Zero). You must write your own when the class owns a raw resource (a raw pointer from new, a file handle, a socket) whose copy has to be made explicitly.

Alternative concept – std::vector<std::string>

```
class Prescription { std::vector<std::string> medicines; public: explicit Prescription(int n)
: medicines(n) {} /* no destructor, no copy ctor, no operator= */ };
```

The compiler-generated copy constructor, assignment and destructor are all correct (Rule of Zero). Time to copy n medicines is O(n) and space O(n) in both designs; the shallow copy is O(1) but incorrect. Measured: the vector copy was 1.4–1.9× faster than my hand-written array (0.074 vs 0.104 µs at n = 10; 599 vs 1,134 µs at n = 100,000), because it constructs each string in place instead of default-constructing n strings and then assigning to them. Code size: 0 extra lines versus about 12.

Q3 – Dynamic Constructors and Destructors (7 marks)

Design – memory layout of the seat map

```
SeatMap object (rows = 3, seats = 4)            heap
  event  : std::string
  rows   : 3                map ---> [ ptr0 | ptr1 | ptr2 ]      (3 x 8 bytes, new char*[3])
  seats  : 4                           |       |       |
  map    : char**                      v       v       v
                                    [. . . .][. . . .][. . . .]   (3 separate new char[4])
Allocations: rows + 1 = 4.  The destructor frees each row with delete[], then the pointer
array with delete[].
```

Implementation

```
SeatMap(const std::string& e, int r, int s) : event(e), rows(r), seats(s), map(nullptr) {
    if (r < 0 || s < 0) throw std::invalid_argument("negative size");
    map = new char*[rows];
    for (int i = 0; i < rows; ++i) { map[i] = new char[seats]; for (int j = 0; j < seats; ++j)
map[i][j] = '.'; }
}
~SeatMap() {
    int booked = 0;  /* count 'X' for the summary */
    for (int i = 0; i < rows; ++i) delete[] map[i];     // every row
    delete[] map;                                       // then the pointer array
    std::cout << "~SeatMap(" << event << ") booked=" << booked << " of " << rows*seats << ",
memory released\n";
}
bool book(int r, int s);  bool cancel(int r, int s);    // range-checked, refuse double
booking
```

Analysis

- **Order of constructors and destructors:** objects are destroyed in the reverse order of construction. A local (stack) object is destroyed when its scope ends; a block-scope object when the block ends; a heap object only when delete is executed on it.

- **delete forgotten:** the destructor never runs, so nothing is released – the object and all of its rows leak. **delete instead of delete[]:** undefined behaviour; the runtime cannot know how many elements to destroy, and the allocator functions do not match.
- **Destructor takes no arguments and cannot be overloaded** because it is called implicitly, by the compiler, at the end of lifetime; there is no place in the program text to supply arguments and the compiler would not know which overload to choose. A class therefore has exactly one destructor.

Experimental investigation (actual results)

Order of construction and destruction (stack, block, heap)

```
 + SeatMap(Stack-A, 2x3)
enter block
 + SeatMap(Block-B, 2x3)
leave block
 - ~SeatMap(Block-B) booked=1 of 6, memory released     <- block object: at the closing brace
 + SeatMap(Heap-H, 2x3)
 (second booking of the same seat refused)
delete h
 - ~SeatMap(Heap-H) booked=1 of 6, memory released      <- heap object: exactly at delete
end of function -> stack object Stack-A is destroyed next
 - ~SeatMap(Stack-A) booked=0 of 6, memory released     <- stack object: at end of its scope
```

Run	Result
-fsanitize=address, delete forgotten (event created in a function, never deleted)	ERROR: LeakSanitizer: detected memory leaks – Direct leak of 16 bytes (the object) + indirect leaks of 24 bytes (pointer array) and 12 bytes in 3 objects (rows). SUMMARY: 52 byte(s) leaked in 5 allocation(s). Note: my first attempt, with the pointer still sitting in main's stack frame, was NOT reported, because LeakSanitizer only reports blocks it can no longer reach.
delete map[i] / delete map instead of delete[]	ERROR: AddressSanitizer: alloc-dealloc-mismatch (operator new [] vs operator delete)
Correct code (delete[] and destructor)	no sanitizer output, exit code 0
Event with zero rows	Created normally (new char*[0] is valid); book(0,0) is refused by the range check – no crash
Event with -1 rows	std::invalid_argument "negative size", no memory allocated (the check precedes the allocation)

Memory requested by new (counted by replacing operator new)

Seat map	new calls	Seat data (chars)	Row pointers (8 B each)	Total bytes requested
10 × 20	11	200	80	280
100 × 200	101	20,000	800	20,800
1,000 × 2,000	1,001	2,000,000	8,000	2,008,000

Predict the Output – Logger

Prediction (hand-traced) and actual output (identical)

```
+A +B -B copyAc use -Ac +P -P end -A
```

Output	Why
+A	Logger a("A") constructed.
+B -B	b constructed inside the block, destroyed at the closing brace.
copyAc use -Ac	use(Logger) takes its parameter BY VALUE: the copy constructor builds a temporary whose name is "A"+"c" = "Ac" (prints copyAc); the body prints use; the parameter is destroyed when the call ends (-Ac).
+P -P	new Logger("P") then delete p: constructor and destructor run at those exact points.
end -A	"end " is printed, then main returns and the stack object a is destroyed (-A). There is no trailing newline because the program never prints one.

Conclusion – RAII

Tying release to the destructor means the cleanup code is written once, in one place, and runs automatically on every exit path: normal return, early return and exceptions (the stack is unwound and destructors run). A forgotten delete or a missed error path can no longer leak, which is exactly what the sanitizer runs above demonstrated by contrast.

Alternative concept – one flat block or vector<vector<char>>

Layout	Allocations (100 × 200, measured)	Access	Extra space
Array of row pointers (this design)	101 (rows + 1)	map[r][s]: two loads, O(1)	100 × 8 = 800 B of pointers (+ malloc overhead per row)
One block, index r*seats + s	1	one multiply-add and one load, O(1)	none
vector<vector<char>>	102 (outer + 100 rows + 1 temporary used to initialise the rows)	v[r][s], O(1)	each inner vector holds 24 bytes of bookkeeping
One vector<char> of r*s	1	v[r*seats+s], O(1)	24 bytes of bookkeeping total

Access is O(1) in all layouts. The flat block needs one allocation instead of rows + 1, so construction and destruction are faster and the data is contiguous (cache friendly); the row-pointer layout is only worth it when rows have different lengths (a jagged array).

Q4 – Overloading Arithmetic and Stream Operators (7 marks)

Design and implementation

```
class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
    Money operator+(const Money& o) const { Money r; r.paise = paise + o.paise; return r; }
// TODO 1
    Money operator-(const Money& o) const { Money r; r.paise = paise - o.paise; return r; }
    Money& operator+=(const Money& o)     { paise += o.paise; return *this; }
    bool operator==(const Money& o) const { return paise == o.paise; }
// TODO 2
    bool operator<(const Money& o)  const { return paise <  o.paise; }
    friend std::ostream& operator<<(std::ostream& os, const Money& m) {
// TODO 2
        long long a = m.paise < 0 ? -m.paise : m.paise;
        return os << (m.paise < 0 ? "-" : "") << "Rs " << a / 100 << '.'
                  << std::setw(2) << std::setfill('0') << a % 100 << std::setfill(' ');
    }
    friend std::istream& operator>>(std::istream& is, Money& m);        // reads
"rupees.paise"
};
```

Actual output of the given program (matches expected output)

```
Rs 200.00
true
```

Analysis

- **+ returns a new object, += returns a reference to *this:** a + b must leave both operands unchanged, so the result is a new Money. a += b modifies a, and returning *this by reference lets chains such as (a += b) += c work without copyin**
- **Members or friends:** + − += == < are members, because they work on *this and one other Money and need no special access beyond their own class. << is a friend (non-member) – see next point. >> is a friend for the same reason.
- **Why << must be a non-member:** in std::cout << m the left operand is the std::ostream. A member operator<< would have to belong to ostream, a library class we cannot modify; so it is written as a free (friend) function taking (ostream&, const Money&), returning the same stream so that << can be chained.
- **How a + b is translated:** the compiler rewrites it as a.operator+(b) for a member, or operator+(a, b) for a non-member; it is an ordinary function call with the operator's name.

Experimental investigation (actual results)

Test	Result
Carry: Rs 0.75 + Rs 0.50	Rs 1.25 (125 paise: the carry into rupees is automatic because only paise are stored)
Negative: Rs 10.00 − Rs 12.50	-Rs 2.50. Decision: allowed and printed with a minus sign – it is a real value (a refund or a debit); the caller can test negative(). A

Test	Result
	wallet that must never go below zero would reject it at the point of use, not inside Money.
m + 100 with member operator+	Rs 100.50 – the int 100 is converted to Money(100) on the right-hand side
100 + m with member operator+	error: no match for 'operator+' (operand types are 'int' and 'Money')
100 + m with friend operator+(const Money&, const Money&)	compiles; prints 10050 for both m + 100 and 100 + m
Reading "45.30" with >>	Rs 45.30

1,000,000 additions of Rs 0.10 – Money versus double

Type	Result	Error	Time
Money (long long paise)	Rs 100000.00	0 – exact	0.34 ms
double rupees	100000.00000133	+1.33 × 10^-6	2.9 ms

The double total is wrong by 1.33 × 10^-6 after only a million additions, because 0.10 is not representable in binary. In this measurement the Money loop was also faster: the dependent chain of integer additions has lower latency than floating-point additions. (A volatile step value was used so that the compiler could not fold the loop into a constant.) sizeof(Money) = sizeof(double) = 8 bytes.

Readability: without operators the bill is print(add(add(a, b), c)); with operators it is std::cout << a + b + c; – the second reads like the arithmetic it performs and cannot be mis-nested.

Engineering decision – should Money allow Money × Money?

No. Rs × Rs would produce "rupees squared", which has no meaning for a bill; allowing it hides errors (multiplying two amounts by mistake) and a long long result would no longer be a Money in paise. Sensible: Money + Money, Money − Money, comparisons, unary minus, Money × number (long long scalar, provided as the friend operator*(long long, Money): 3 × Rs 12.50 = Rs 37.50 in the test), and Money ÷ number with an explicit rounding rule. Meaningless: Money × Money, Money + plain number without a unit, ++/-- (a "next rupee" has no business meaning).

Alternative concept – friend (non-member) operators or named functions

Time and space stay O(1) and 8 bytes in all versions (member, friend, add()/equals()); inlined they compile to the same instructions. The friend form operator+(const Money&, const Money&) allows **both** m + 100 and 100 + m, because implicit conversion applies to both arguments; a member operator+ only converts the right-hand operand (the compile error above). Named functions allow neither natural syntax nor chaining in one expression.

Q5 – Overloading ++, Comparison and [] (6 marks)

Implementation

```
class Time {                                    // h:m:s, rolls over at 24 h
```

```
    int h, m, s;
public:
    Time& operator++()    { if (++s == 60) { s = 0; if (++m == 60) { m = 0; if (++h == 24) h =
0; } } return *this; }  // prefix
    Time  operator++(int) { Time old = *this; ++*this; return old; }
// postfix
    bool operator==(const Time& o) const { return h == o.h && m == o.m && s == o.s; }
    bool operator<(const Time& o)  const { return h != o.h ? h < o.h : m != o.m ? m < o.m : s
< o.s; }
};
class Playlist {
    std::vector<std::string> tracks; std::string dummy = "<invalid>";
public:
    std::string&       operator[](int i);          // checks 0 <= i < size, prints an error,
returns dummy
    const std::string& operator[](int i) const;    // same check, read-only access
};
```

Analysis

- **Prefix versus postfix:** both are named operator++. The postfix version has an extra, unnamed int parameter that is never used; it exists only so the compiler can tell the two apart: ++t calls operator++(), t++ calls operator++(0).
- **Why postfix returns a copy and is slower:** it must return the value from BEFORE the increment, so it saves a copy of the old object, increments, then returns the saved copy by value. Prefix just increments and returns a reference – no copy.
- **Const and non-const operator[]:** the non-const version returns a modifiable reference (playlist[0] = "Jai Ho"). A const Playlist (for example a const Playlist& parameter) can only call const member functions, so a const overload is needed to read from it; it returns a const reference so the element cannot be changed through it.
- **Out-of-range index:** the program must not touch memory outside the vector (undefined behaviour). Here an error message is printed and a harmless dummy element is returned; an alternative is to throw std::out_of_range (see the at() alternative).

Experimental investigation (actual results)

```
t starts 0:00:10;  a = t++ ;  b = ++t
  a=0:00:10  b=0:00:12  t=0:00:12   (a is the OLD value, b the NEW)
roll-over: 0:59:59 -> 1:00:00   23:59:59 -> 0:00:00
t1 == t2 ? true;  0:59:59 < 1:00:00 ? true
playback simulation:
  0:00:00  playing "Vande Mataram"
  0:00:03  playing "Chaiyya Chaiyya"
  0:00:06  playing "Kun Faya Kun"
  non-const write p[0] = "Jai Ho" -> Jai Ho
  const access p[1] = Chaiyya Chaiyya
test p[-1]:
  [error] index -1 out of range 0..2
test p[size] (=3):
  [error] index 3 out of range 0..2
at(3) alternative throws: vector::_M_range_check: __n (which is 3) >= this->size() (which is
3)
```

10,000,000 increments and accesses

Measurement	-O0	-O2
Prefix ++t	31.3 ms	5.3 ms
Postfix t++ (result used)	70.9 ms	24.8 ms
Checked playlist[i]	84.7 ms	14.9 ms
Unchecked vector[i]	46.7 ms	7.4 ms

Postfix is 2.3× (-O0) and 4.7× (-O2) slower than prefix here, even for a 12-byte Time; for a large object the copy would cost far more. The range check adds about 7 ms per 10 million accesses at -O2 (≈ 0.7 ns per access), the price of one extra comparison and the wrapper call.

Predict the Output – Counter

```
5 7 7
```

c.v starts at 5. Counter a = c++ (postfix): the old object (v = 5) is copied into a, then c becomes 6. Counter b = ++c (prefix): c becomes 7 and b copies the new value 7. So a.v = 5, b.v = 7 and c.v = 7. The postfix version made a copy of the object (Counter old = *this); the prefix version did not.

Conclusion – why prefer ++i to i++ for class types?

When the old value is not needed, ++i does the same work without creating, copying and destroying a temporary object – measured above as 2.3–4.7× faster for a tiny class and a larger penalty for objects that own memory. For built-in int the optimiser removes the difference, so the habit costs nothing and pays off with iterators and other class types.

Alternative concept – at() that throws, or an unchecked operator[]

Time per access is O(1) in all three. The checked operator[] adds one comparison pair (measured 14.9 ms vs 7.4 ms per 10 million accesses at -O2, ≈ 0.7 ns each); at() adds the same check but reports the error with an exception that the caller must catch, so an invalid index cannot be silently ignored; the unchecked version is fastest but turns an out-of-range index into undefined behaviour (memory corruption or a crash). For a music player, where a bad index comes from user input, checking is worth the 0.7 ns.

Q6 – The Rules of Operator Overloading: a Text Class (7 marks)

Implementation

```
class Text {
    char* d; size_t n;
public:
    Text(const char* s = "");                                   // allocate and copy
    Text(const Text& o);                                        // deep copy
    Text& operator=(const Text& o);                             // guard self-assignment, new
buffer first
    ~Text();                                                    // delete[] d
    friend Text operator+(const Text& a, const Text& b);        // concatenation: new object of
n1+n2 chars
    bool operator==(const Text& o) const;  bool operator!=(const Text& o) const { return
!(*this == o); }
```

```
    friend std::ostream& operator<<(std::ostream& os, const Text& t);
};
```

Analysis

Operator that cannot be overloaded	Reason
:: (scope resolution)	It names a class or namespace, not a value; its left operand is a name, not an object.
. (member access)	Its right operand is a member name, not a value; overloading it would make the meaning of every member access ambiguous.
.* (pointer to member)	Same reason as the dot operator.
?: (conditional)	It has three operands and evaluates only one of them (short-circuit); a function call would evaluate all arguments.
sizeof	Evaluated at compile time on the type; it must keep one fixed meaning.

- **Rules:** the precedence and associativity of an operator cannot be changed; the number of operands (arity) cannot be changed; no new operator symbols can be invented; and at least one operand must be a user-defined type, so int + int cannot be redefined.
- **operator= must be a member function** (the language requires it) so that it is always tied to its own class and the compiler's implicit assignment can never be bypassed by a non-member.
- **When an operator should be a friend:** when the left operand is not an object of the class (std::cout << t, or 5 + money), or when the operation needs private data of both operands and symmetric conversion of both arguments.

Experimental investigation (actual results)

Experiment	Observed
a + b + c with "SMS: ", "meet at ", "5 pm"	= "SMS: meet at 5 pm"; copy-constructor calls = 0, buffer allocations = 2 (a+b creates one temporary, +c creates the result). With -fno-elide-constructors (copy elision off) the same expression made 2 copy-constructor calls and 4 allocations: compilers normally remove those copies.
Empty messages	size 0; empty + empty = size 0; "x" + empty = "x"
Long message (100,000 characters)	L + L has size 200,000 and equals a Text built from 200,000 chars; L != L2 is true
Self-assignment s = s	still "self" – the this != &o guard works
-fsanitize=address,undefined	exit code 0, 0 errors reported (no leak, no overflow)

100,000 concatenations of one character (final length 100,000)

Version	Time	Complexity of the loop
Text: t = t + Text("x")	298 ms	$O(n^2)$: every + copies the whole string

Version	Time	Complexity of the loop
std::string: s = s + "x"	138 ms	O(n²), but its rvalue operator+ reuses the temporary's buffer
std::string: s += "x"	0.45 ms	O(n) overall: amortised O(1) append with capacity doubling
By-value operator+(T a, const T& b) { a += b; return a; } on a std::string-backed class	301 ms	O(n²), no better than mine

Conclusion

Writing Text taught me what std::string hides: who owns the buffer, why a copy constructor, assignment operator and destructor must agree (the Rule of Three), why self-assignment needs a guard, and why a + b + c creates temporaries. std::string was about 2× faster for s = s + "x" and about 670× faster for s += "x" in the loops above (capacity growth, in-place append, small-string optimisation). I would still write such a class when I need control that the library does not offer – a fixed-capacity message in embedded or SMS firmware (no heap), a special encoding, or a learning exercise – and I would otherwise use std::string.

Alternative concept – std::string, or operator+ implemented with += on a by-value copy

For strings of lengths n1, n2, n3, my a + b + c copies (n1+n2) characters for the first + and (n1+n2+n3) for the second: 2n1 + 2n2 + n3 characters, with a temporary of n1+n2 characters. The by-value version copies a (n1) into its parameter and then appends b (another n1+n2 if the buffer has to grow), so it copies at least one extra n1 and has the same O(n1+n2+n3) order. Measured: 301 ms versus 298 ms for the 100,000-step loop. std::string adds rvalue overloads that move from the temporary, so (a + b) + c appends c into the temporary's buffer without a second full copy.

Q7 – Design a Complex-Number Class from First Principles (10 marks)

Stage 1 – Interpret the problem (2)

- **Data:** a complex number = real part and imaginary part (two doubles, 16 bytes). **Operations:** +, −, ×, ÷, ==, printing in rectangular form, magnitude (for |Z|).
- **Constructors needed:** default Complex() = 0 + 0j (arrays, uninitialised variables); from a real part Complex(r) = r + 0j (a resistor, and mixing Complex with plain numbers); from real and imaginary parts Complex(r, i) (inductors and capacitors).

#	Measurable requirement
R1	Resistor R + 0j, inductor 0 + jωL, capacitor 0 − j/(ωC) are constructed exactly.
R2	Series Z = Z1 + Z2 + … is correct to 3 decimal places against hand calculation.
R3	Parallel Z = (Z1 × Z2)/(Z1 + Z2) is correct to 3 decimal places against hand calculation.
R4	Output prints "a + jb" / "a − jb" and |Z| to 3 decimals.
R5	Division by a zero impedance raises a catchable error instead of printing inf or nan.

Stage 2 – Design (3)

Operator	Formula (a + jb, c + jd)	Member or friend?
+	(a + c) + j(b + d)	member (const, returns new object)
−	(a − c) + j(b − d)	member
×	(ac − bd) + j(ad + bc)	member
÷	((ac + bd) + j(bc − ad)) / (c² + d²); throws std::domain_error if c² + d² = 0	member
==	a = c and b = d	member
<<	prints "re + j|im|" or "re − j|im|"	friend (left operand is ostream)
magnitude()	√(a² + b²) using std::hypot (avoids overflow)	member function

```
class Complex {
    double re, im;
public:
    Complex() : re(0), im(0) {}
    Complex(double r) : re(r), im(0) {}          // real only (implicit, so Complex + 5.0
works)
    Complex(double r, double i) : re(r), im(i) {}
    double magnitude() const { return std::hypot(re, im); }
    Complex operator+(const Complex& o) const { return {re + o.re, im + o.im}; }
    Complex operator-(const Complex& o) const { return {re - o.re, im - o.im}; }
    Complex operator*(const Complex& o) const { return {re*o.re - im*o.im, re*o.im + im*o.re};
}
    Complex operator/(const Complex& o) const {
        double d = o.re*o.re + o.im*o.im;
        if (d == 0.0) throw std::domain_error("division by zero impedance");
        return {(re*o.re + im*o.im)/d, (im*o.re - re*o.im)/d};
    }
    bool operator==(const Complex& o) const { return re == o.re && im == o.im; }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z);
};
Complex parallel(const Complex& a, const Complex& b) { return (a * b) / (a + b); }
```

Stage 3 – Implement and verify (3)

Test circuit: R = 100 Ω, L = 0.1 H, C = 10 µF, ω = 500 rad/s. So XL = ωL = 50 Ω and XC = 1/(ωC) = 1/(500 × 10×10^-6) = 200 Ω, giving ZR = 100 + j0, ZL = 0 + j50, ZC = 0 − j200.

Quantity	Hand calculation	Program output	Match
Series Z	100 + j(50 − 200) = 100 − j150	100.000 − j150.000	yes
Series |Z|	√(100² + 150²) = 180.278 Ω	180.278 Ω	yes
Parallel 1/Z	1/100 + 1/(j50) + 1/(−j200) = 0.01 − j0.02 + j0.005 = 0.01 − j0.015	–	–
Parallel Z	(0.01 + j0.015)/0.000325 = 30.769 + j46.154	30.769 + j46.154	yes
Parallel |Z|	55.470 Ω	55.470 Ω	yes
(3+4j)(1−2j)	3 − 6j + 4j − 8j² = 11 − 2j	11.000 − j2.000	yes

Quantity	Hand calculation	Program output	Match
(3+4j)/(1−2j)	(3 − 8 + j(4 + 6))/5 = −1 + 2j	−1.000 + j2.000	yes

Division by a zero impedance

```
Complex z = Complex(1, 1) / Complex(0, 0);
-> caught std::domain_error: division by zero impedance
```

The parallel of a resistor and a short circuit (Z2 = 0) returns 0 + j0 (correct: the short dominates), while the parallel of two zero impedances has (Z1 + Z2) = 0 and raises the same exception, because 0/0 is undefined.

Stage 4 – Defend the design (2)

- **Comparison with std::complex<double>:** for the same circuit, std::complex gives series (100, −150), |Z| = 180.278 and parallel (30.769, 46.154), |Z| = 55.470; the difference from my class is 0.000 in both cases. sizeof is 16 bytes for both. std::complex has more functions (polar form, exp, sqrt, arg) and is tested on every platform.
- **Why each constructor and operator is needed:** the default constructor lets arrays and variables start at 0; Complex(r) lets resistors and plain numbers mix with Complex; Complex(r, i) builds inductors and capacitors; + for series; × and ÷ for the parallel formula; − for differences of voltages; == for tests; << for the required output; magnitude() for |Z|.
- **Division by zero:** my class throws a catchable std::domain_error. std::complex does not: (1+j)/0 produced (inf, inf) silently, which would propagate into later results – the program has to test for it explicitly.
- **One remaining limitation:** because the arithmetic operators are members, Complex + 5.0 works but 5.0 + Complex does not (the left operand is not converted); they should be friends. Also == compares doubles exactly, which is unreliable after calculations (a tolerance is needed), and the zero test d == 0.0 will miss values that are almost zero.

Alternative concept – use std::complex<double>

Time per operation is O(1) and space is two doubles (16 bytes, measured by sizeof) for both. Accuracy was identical to three decimals in this circuit (difference 0.000). The work needed: with std::complex there is no class to write or test, but division by zero must be handled by the caller (check the denominator before dividing, or test for isinf/isnan afterwards); in my class it is built in once and cannot be forgotten.

Simulation Screenshot

Real terminal run (g++ -std=c++17): the constructors, the deep-copy program under AddressSanitizer, the sanitizer error for the version without a copy constructor, the construction/destruction order, the Logger and Counter predictions, the Money program and the RLC impedance results.

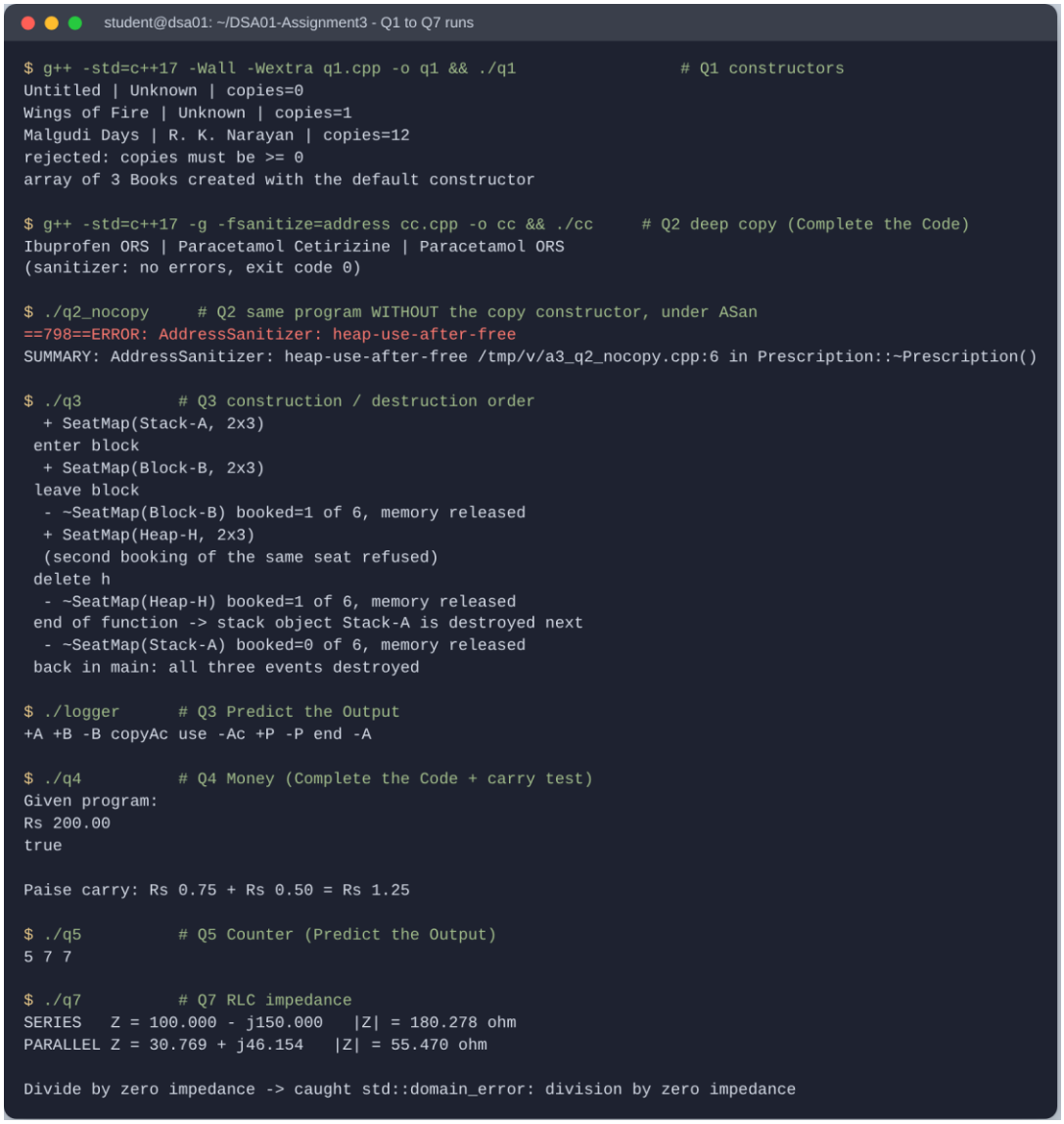


Figure 1 – Terminal run of the Assignment 3 programs.